The Use and Implementation of Contracts

ISO/IEC JTC1 SC22 WG21 EWG P0147R0 - 2015-11-08

Lawrence Crowl, Lawrence@Crowl.org

Introduction

Contracts are the preconditions, invariants and postconditions that define the behavior of a software abstraction. The common practice is that these contracts appear in vague documentation, if at all. The standard library uses a much more formal documentation to define behavior. That documentation is critical to interoperable software. However, that documentation is in prose, not code.

Moving contracts from prose to code has several benefits.

Contracts in code enables automatic checking of correctness.

The checking can occur at run time via inserted check code. It can also occur at compile time via static analysis. The simple case for static analysis is determining that a constant value does not satisfy a condition. A more involved static analysis could extend to finding that the postcondition that produced a value is weaker than the precondition that uses the value.

Contracts in code enables more optimization of programs.

A contract constrains the behavior of a function, and compilers exploit constraints. As examples, a precondition that states a parameter is not null can enable the compiler to remove checks for null pointers. Similarly, an invariant that a loop bound is 0 mod 4 enables the compiler to avoid emitting tail code when unrolling loops.

Contracts in code encourages better contracts.

Specifications generally become less ambiguous and more complete as they become more formal. Furthermore, programmers tend to question the correctness and completeness of code much more than they do of prose. Taken together, these points mean that contracts in code will lead to fewer misunderstandings and defects,

particularly for programmers new to the code.

While we could realize many of these benefits for the standard library by baking knowledge of the library into the compiler, the approach would provide no value to other libraries. We need a general solution.

This paper does not propose a specific solution. It does use a syntax somewhat close to that proposed in N1962 Proposal to add Contract Programming to C++. For specific proposals and other discussion, see the Bibliography below.

Instead, this paper explores the writing of contracts with some examples, explores the practical issues in managing of contract compilation, and outlines an implementation that addresses those issues. In the process, some constraints arise on the semantics of any specific contract proposal.

Writing Contracts

All contracts start life as a vague goal, which is then refined to a usable contract. Writing a refined contract can be hard, whether or not the contract is in prose or in code. In this section, we show through a sequence of examples how that refinement may happen. In the process, we identify constraints on a contract mechanism. The working assumption in this section is that the goal is a complete contract in code. Where completeness in code is not possible or desirable, the coded parts of the contract remain valuable.

Preconditions on Arguments

Preconditions generally describe the domain of the computation. The (real-valued) square root function has a well-known domain.

```
double sqrt(double input)
  precondition { input >= 0.0; };
```

However, IEEE floating-point [IEEE] numbers are not real numbers. Any contract on a IEEE floating-point numbers must take into account the 'extra' values. Among them are negative zero, infinity and not-a-number (NaN).

```
double sqrt(double input)
  precondition { ! isSignMinus(input); input < +Infinity; };
```

The [IEEE] operation `isSignMinus` tests the sign bit, enabling one to distinguish between positive zero and negative zero. So, the first condition excludes all negative values. The second condition excludes positive infinity. It also excludes NaNs because NaNs are never less than any other value.

In any event, **preconditions must identify the subset of possible input values that constitutes the domain of the function**.

Postconditions on Return Values

Postconditions define the return value of a function given that the preconditions have been met.

```
vector<int> sort(vector<int> arg)
  postcondition { is_sorted(result); };
```

While necessary, the above postcondition is not sufficient. It must also say that the output is a permutation of the input.

```
vector<int> sort(vector<int> arg)
  postcondition { is_sorted(result); is_permutation(arg, result); };
```

So, **postconditions must describe the relationship of the result to the values in the domain of the input**.

Chaining Conditions

In general, we satisfy the preconditions of one function with the postconditions of earlier function. Indeed, such chaining is unavoidable if we consider primitive operations as functions with contracts.

In the example above, the `is_sorted` condition would *a priori* satisfy an `is_sorted` precondition that `is_permutation` might have on its second argument.

The chaining of postconditions to preconditions implies that both are inherently part of the interface to a function. So, **preconditions and postconditions should be visible as part of the function header**.

Postconditions as Inverse Functions

The obvious postcondition for `sqrt`, and the one that prose documentation would typically state, does not work.

```
double sqrt(double input)
  precondition { ! isSignMinus(input); input < +Infinity; };
  postcondition { result == sqrt(input); };
```

The problem is that `sqrt` is using itself to define itself. Instead, **postconditions should often be specified in terms of the inverse function**.

```
double sqrt(double input)
  precondition { ! isSignMinus(input); input < +Infinity; };
  postcondition { result >= 0.0; result*result == input; };
```

Completeness via Code

Unfortunately, the above `sqrt` specification ignores the rounding inevitable with a discrete representation of real numbers. This lack of completeness arises because **programmers are less likely to notice incompleteness in prose specifications**. For completeness, the `sqrt` specification needs to define how close the result must to the true square root. One approach is the following.

```
double sqrt(double input)
  precondition { ! isSignMinus(input); input < +Infinity; };
  postcondition { result >= 0.0;
                  abs(result*result-input) < input*1e-16; };
```

This specification is weaker than typical guarantees, which require "as close as is representable". For that, we introduce a helper function to determine if the result is close enough. Central to this determination is the use of [IEEE] operations `nextUp` and `nextDown` which provide access to the closest representable number. In essence, we define the result to be within the range implied by the closest computations. (One can, of course, generalize the 'close enough' function to accept an arbitrary function. Such a function would be useful in the specification of a variety of contracts.)

```
bool sqrt_close_enough(double input, double result) {
  if ( input < result * result )
    return result * max(nextDown(result), 0.0) <= input;
  else if (result * result < input )
    return input <= result * nextUp(result);
  else
    return result * result == input;
}
```

Now we can provide a more complete contract.

```
double sqrt(double input)
  precondition { ! isSignMinus(input); input < +Infinity; };
  postcondition { result >= 0.0;
                  sqrt_close_enough(input, result); };
```

The use of helper functions is an important tool in the specification of contracts. Among other things, they provide the compiler with a handle to match preconditions with postconditions.

Given user-control of rounding modes, more specification is needed. At this point, one is tempted to say "too much work"! However, failing to do such work when writing the specification simply defers the work to support calls.

Wide versus Narrow Contracts

The contract for `sqrt` presented above is *not* what [IEEE] requires. The IEEE contract has no preconditions. Instead, it specifies the behavior for all possible input. In the terminology of John Lakos, the IEEE `sqrt` has a wide contract.

In effect, the nominal preconditions have been moved to the postcondition as exceptional values.

```
double sqrt(double input)
  postcondition {
    if ( isNaN(input) || input < 0.0 ) isNaN(result);
    else if ( input == 0.0 && isSignMinus(input) )
           result == 0.0 && isSignMinus(result);
```

```
    else if ( input == +Infinity ) result == +Infinity;
    else sqrt_close_enough(input, result); };
```

This approach follows the common practice of not failing on any possible input. The approach is a consequence of having of no contract mechanism in the language and of a desire of programmers not to have 'their' function fail. Unfortunately, the approach usually does not eliminate the failure, it simply defers it to later in the execution, where the root cause is harder to discern. So usually, **the precondition should restrict the domain to the region where the function can provide a service**.

Given the above discussion, should we conclude that [IEEE] is wrong? No, we should not. First, an intermediate computation with bad data may not affect the final result. The design chosen enables irrelevant bad computation to disappear. Second, floating-point computation is performance-sensitive. An ability to stop a computation given a broken precondition would likely reduce the performance of hardware and hence applications. Given those observations, the designers felt that the better approach would be for programmers to check a computation's result for validity and redo a failing computation with another algorithm.

Floating-point computation is not the only scenario in which preconditions may be inappropriate. In particular, **preconditions on data arriving from input are fragile**. The problem is that the data may be produced in an environment that is ignorant of the conditions on that data. The better approach is to explicitly test for inappropriate data and report the problem to the user.

Given that wide contracts need no preconditions, we can conclude that **the purpose of preconditions is to define narrow contracts**. Furthermore, a very narrow contract will throw no exceptions, and hence **contracts enable noexcept functions**.

Broken Contracts

An important question to consider is the semantics of a broken contract. The goal of a contract mechanism is to enable programmers to develop programs that are correct and hence do not need checking for correctness. When checking is not enabled, a broken contract can lead to memory corruption and more.

When checking is enabled, what happens when a broken contract is detected? We must to something, and that something is defined by a handler. That handler is generally not defined by the programmer that writes a contract. Indeed, the contract programmer usually has no idea what a handler may do.

A consequence of the above discussion is that **a postcondition should *not* make any assertions about results when preconditions are false**. Doing so would imply that preconditions are weaker in practice than in definition.

As a consequence, the unofficial contracts study group has a working definition something along the lines of "the program has undefined behavior on exit from a broken contract". This statement is true even if the contract is not evaluated.

There is a critical distinction to be made here between the standard not defining behavior and the compiler not defining behavior. Compiler vendors are free to define behavior where the standard does not. Indeed, lack of defined behavior in the standard enables many different behaviors in the compiler, which enables compiler vendors to provide options to handle broken contracts in a way that suits the needs of the user at the moment.

Daniel Garcia suggested that we could define a condition to throw when it was broken. This behavior could then be exploited by programmers calling the function out of contract to generate an exception. Such a reaction is perverse, but perverse things sometimes happen in real code. More importantly, **conditions with defined exception semantics are not different from a wide contract**. We should not introduce a new mechanism for what we can already say.

At the October 2015 standards meeting, a suggestion was made that some programs must never stop, even in the presence of broken contracts. However, for such critical non-stop software, other problems presently exist that make the goal difficult to achieve. Memory access violations from wild pointers, dereferencing a null pointer, and other core-language features result in instant termination. These features all have narrow contracts; the program is assumed to be well-behaved. Achieving a non-stop goal requires either a proof of correctness, or a program that consists of only wide contracts. The standard library provides neither. With contracts though, the standard library could provide assurance through rigourous testing. In any case, requiring broken contracts to not terminate the program does not solve enough of the problem to justify its definition. Instead, programmers should rely on compiler-defined behavior in engineering their programs.

Conditions on Object Identity

We know we get a non-null pointer from `c_str`.

```
const charT* basic_string::c_str()
  postcondition { result != nullptr; };
```

However, the relationship of that pointer to extant objects in the program is important. The standard's `c_str` postcondition

describes that relationship. (There may be a better way to say this postcondition, but it is the standard's approach.)

```
const charT* basic_string::c_str()
  postcondition { result != nullptr;
                  for ( size_type i = 0; i <= this->size(); ++i )
                     p+i == &this->operator[](i); };
```

So, **conditions can say when two objects are the same**.

Just as importantly, we can **use conditions on object identity to disallow aliasing**. For example, we can disallow self-assignment.

```
T& operator=(const T& arg)
  precondition { &arg != this; };
```

The `memcpy` function is a more complex example of aliasing.

```
void* memcpy(void* dst, const void* src, size_t len)
  precondition { (const char*)dst + len < (const char*)src ||
                 (const char*)src + len < (const char*)dst; );
```

Conditions on Object State

The domain of a computation can include the state of an object it accesses. To avoid leaking implementation artifacts into the contract, only public functions should appear in a precondition.

```
T& deque::front()
  precondition { this->size() > 0; };
```

Likewise, one must be able to assert state in the postcondition.

```
T& deque::push_front(const T& x)
  postcondition { this->size() > 0; };
```

An important implication of this approach is that **abstraction designers must expose enough public state-querying functions to define the contract and to enable clients to meet the contract**.

Invariants

Invariants specify that a certain state is expected at a given point in the code. Invariants are helpful in developing and analyzing code. Loop invariants are helpful in part because they provide a statement of an inductive step in reasoning. Furthermore, invariants help drive proofs of correctness for applications that need a rigorous demonstration of quality.

The most widely known invariant mechanism is the `assert` macro in C. The `assert` macro appears in block scope. We call this a block-scope invariant. The primary uses of block-scope invariants are with respect to the *implementation* of an abstraction, rather than to the *interface* of an abstration. As such, **block-scope invariants are not properly part of the contract**. However, as the mechanisms and effects are similar, we consider them part of the same language facility.

Despite the above assertion, John Lakos points out a use for internal invariants in precondition checking. His example is a search into a sorted array. The normal precondition would be that the array is sorted. This would take time linear in the size of the array. An alternate checking strategy is to add consistency check into the search procedure itself. This strategy does not check the entire array; it only checks that portion of the array that it visits. This strategy has log complexity, just as the search itself does, a substantial savings.

Invariants can also appear at class scope. The main difference is that such invariants can make claims about the state of a class object between all operations. The invariants are established on completion of the constructor and must be valid on entry to the destructor. The compiler would be responsible for inserting checking code in all (non-private) functions that might establish, use, or modify the state. That is, **a class-scope invariant is the part of a contract that specifies what it means for an object of the class type to be well-formed**.

Likewise, namespace-scope invariants specify the state of a namespace-scope variable between operations on it. Namespace-scope invariants differ from class-scope invariants in that they make claims about variables, rather than types. So, **a namespace-scope invariant is the part of a contract that specifies what it means for a namespace-scope variable to be properly used**.

Contracts and Exceptions

The specification of behavior when exceptions occur within contract evaluation would be extremely difficult to get right. Moreover, they would likely be difficult to understand. So, **an exception within contract evaluation breaks the contract**. An implication is that **contracts should first test for conditions that might later lead to exceptions**.

When a function throws an exception, it almost certainly could not do what it was asked to do. In this case, the postconditions are unlikely to be met. So, **an exception from a function call nullifies the postcondition**.

On the other hand, class-scope invariants apply regardless of the manner of member function exit. Therefore, **class-scope invariants define the minimal meaning of exception safety.**

Some operations make stronger exception safety claims. In particular, they claim that an exception thrown from a member function will leave the state of an object unchanged from its state on function entry. **If we invalidate postconditions on exceptions, we appear to have no mechanism to describe strong exception safety.**

Inheritance

In the presence of inheritance, we need to define the relationship of contracts between members of an inheritance hierarchy.

Consider a class `B` with preconditions and postconditions on its member functions. (For the rest of this section, we use "...conditions on `B`" to mean "...conditions on the member functions of `B`.) Now consider a class `D` derived from `B` and intended to be used in the place of `B`. The preconditions on `D` can be no stronger than those on `B` because clients of `B` would have no knowledge of those additional requirements. The preconditions on `D` could be weaker than those on `B`. The postconditions on `D` can be no weaker than those on `B` because clients of `B` would have no knowledge of those weaker results. The postconditions on `D` could be stronger than those on `B`.

So, **inherited preconditions only get weaker while the inherited postconditions and invariants only get stronger**. Eventually, one gets to an invariant constant function, which requires no preconditions and leaves room for a vast number of predicates on a known value. There is something fundamental here, but identifying it is left as an exercise for the reader.

Writing Expectations

Correctness requires knowing what must be true. Performance depends only on knowing what is usually true. We call statements on what is usually true an *expectation*.

Optimizing Branches

When compilers know the common case in a branch, they can optimize the code to deliver better average-case performance. Compilers can discover the common case by instrumenting programs to collect an execution profile. Unfortunately, programmers resist using such capabilities because of concerns of build complexity, build time, and training set fidelity.

To avoid profiles, compiler vendors often provide extensions to enable programmers to state their expectations about likely program behavior. The most widely known version is probably `__builtin_expect` [GCC]. Such extensions enable the compiler to optimize for the common case.

These extensions are are unique to each compiler, which inhibits their use to only highly performance-sensitive programs and which inhibits porting of the resulting code. A standard syntax would solve both problems. N4425 Generalized Dynamic Assumptions addresses this concern, but with no integration with other uses of expressions to describe program behavior.

Selecting Algorithms

While a standard optimization syntax would be helpful, it is not the most valuable part of an expectation. **The primary value in an expectation is in communcating the intended operating environment of an abstraction.** For example,

```
void sort1(vector<int> arg)
  expectation { arg.size() <= 7; };
```

both tells when use is appropriate and gives reason to use a bubble sort.

Another example is the multiplication of large numbers.

```
bignum multiply1(bignum a, bignum b)
  expectation { (-1<<60) < a && a < (1<<60);
                (-1<<60) < b && b < (1<<60); };
```

```
bignum multiply2(bignum a, bignum b)
  expectation { a < (-1<<600'000) || (1<<600'000) < a;
                b < (-1<<600'000) || (1<<600'000) < b; };
```

For `multiply1`, the classic multiplication algorithm will likely yield the best performance. For `multiply2`, an algorithm based on the Fast Fourier Transform will likely yield the best performance.

Going further, one can imagine a profile-based analysis suggesting alternate functions whose expectations better match the actual behavior of the program.

A consequence of expectations in the function interface, as opposed to just at block scope, is that **expectations are a variant of preconditions, postconditions and invariants**. They are not an independent feature.

Managing Contracts

In correct code, contracts need not be checked because the code will meet its conditions. However, code starts incorrect and is refined to correctness. One purpose of contract support in the language is to speed that refinement.

Compiling Contracts

At different stages in refinement, programmers will require different amounts of checking. During initial stages of development, programmers will want very aggressive checking to help them identify errors early and quickly, even at the cost of substantial compile-time and run-time overhead. At release, programmers will want minimal checking, preferably at very low overhead.

Everyone anticipates a set of 'build modes' to control what is and is not checked. The issue is who decides what gets checked. The 'who' implies a 'when'. The 'what' implies knowledge of the character of conditions.

Who Makes the Decision?

In deciding on what gets checked, we must first understand who has responsibility for what. The implementor of an abstraction is reponsibile for satisfying the invariants and postconditions *given* that the preconditions have been satisfied. The user of an abstraction is responsible for satisfying the preconditions *given* that the postconditions of prior computations have been satisfied.

If the goal of contracts is to help programmers produce better code, **the control over checking of a condition should reside with programmer who can modify the code leading to that condition**. That is, implementors of an abstraction should control checking of invariants and postconditions while users of an abstraction should control checking of preconditions.

When is the Decision Made?

Consider the case of a non-inline, non-template function. The implementor of the function will compile the function, which implies that the compiler options should control the checking of the invariants and postconditions of the function. More importantly, it implies that the compiler options should control the checking of the preconditions of the other functions called by the function.

The easiest way to reach this compilation-control goal is to adopt a policy where **precondition evaluation the responsibility of the calling function**, and not of the called function. Doing so means that all conditions that can be satisfied by modifications the function are controlled by the compilation of that one function.

What Contracts are Affected?

The decision on what contracts to evaluate depends on there characteristics of a contract: exposure, importance, and cost.

exposure

> A function in the API of a major component is more exposed to naive users than is a function that is only internal to a component. Being more exposed, the API function will induce a desire for more defensive programming, and hence more aggressive evaluation of preconditions.
>
> The exposure of a function is generally obtainable from existing sources, e.g. scope and escape analysis, and so the contract characterization mechanism need not provide any information on it.

importance

> In a sense, all conditions are important because violation of any of them may lead to undefined behavior. However, in practice, some conditions are more likely to catch errors. Of course, if a condition never catches errors, it is implied by

another condition and is ineffective. Furthermore, some conditions are more likely to lead to permanent data corruption. John Lakos gave an example of writing a bad date to a database. Once corrupt data enters the database, it is extremely hard to identify and correct. So, we need a mechanism to characterize the importance of a condition.

cost

Evaluating a condition has a cost. The primary factor in the cost is its computational complexity. This complexity is difficult for the compiler to discern, and so we need a mechanism to characterize it.

The importance of a condition is theoretically independent of its cost. However, if a condition is critical, its cost is moot. Even so, programmers would be well-advised to keep critical conditions cheap.

So, **a contract mechanism should characterize conditions by importance and cost**.

In practice, compilation of contracts will driven by compiler options. Those options may consider factors other than those provided directly in the characterization mechanism, which enables more nuanced management while keeping the characterization mechanism simple.

Characterizing Contracts

The above discussion will doubtless cause concern over the complexity of characterizing contracts. A mechanism that is too complex will not gain support. A mechanism that is too simple will not solve the problem. Moreover, we must keep in mind that ambiguity from a 'simple' mechanism may well introduce more complexity in practice because different programmers will use the same mechanism for different purposes.

We can avoid substantial complexity by ensuring that the contract characterization mechanism is a description of data, not policy or intended use.

Importance

Some conditions will be 'critical', and we can expect them to evaluated even in performance-sensitive code. Some conditions are 'effective' in debugging but are unlikely to have significant consequences if they are not met in production. Somewhere in the middle are conditions that are just 'important'.

We can probably simplify the importance of a condition to 'critical or not'.

Cost

Evaluating a condition has a cost. There are at least two approaches to describing that cost, each leading to several different descriptions.

What is the absolute cost of the condition relative to machine instructions?

1. constant complexity word operations
2. constant complexity function calls
3. log complexity queries on data structures
4. linear complexity on 'small' parameters, e.g. string comparison
5. linear complexity on data structures
6. linear-log complexity on data structures
7. polynomial complexity on data structures
8. exponential complexity on data structures

What is the cost of the condition relative to the function?

1. less complex than the function
2. equally complex as the function
3. more complex than the function

We can combine the two approaches. We can also simplify the cost of a condition to cases where its effect on overall execution is significant.

1. constant complexity
2. equally or less complex than the function
3. more complex than the function

Combined and Simplified

Given the above discussion, consider the following mutually exclusive contract characterizations of a condition.

critical importance

: The programmer of a function decides when a condition is critical. E.g. John Lakos's example of a function writing ill-formed data to files.

constant complexity

: The condition has constant complexity, e.g. primitive operations or very short string comparisons. Evaluating such conditions will have the least impact on execution time.

no more complex

: The condition is no more complex than the function itself. Evaluating such conditions will not increase the complexity of the program as a whole.

more complex

: The condition is more complex than the function itself. Evaluating such conditions will increase the complexity of the program as a whole.

Note that these characterizations do not imply a build mode. However, they do have an order of selection that serves as data in the build mode.

Adopting Contracts

Once C++ supports contracts, we can expect programmers to adopt contracts into working programs.

Adopting contracts into a program is an incremental process. Therefore the addition of a contract should not have ripple effects through the program, as 'const poisoning' did in the past. Therefore, **adding a contract should not change type signatures or noexcept properties**.

This statement means either that preconditions and their handlers must not throw or that preconditions and handlers can throw only in throwing functions. The latter statement is not viable because the programmer that defines a function does not know what handlers might be employed and the programmer that installs a handler does not know all the affected functions. Therefore, **the standard most not define behavior for a condition or handler that throws an exception**. Compilers may define such behavior, but exploiting that behavior should be used for specific purposes and should not be considered generally portable.

We can exploit compiler-defined behavior for broken contracts when adopting contracts into a working program. The steps for adoption are as follows. Not all portions of a program need be at the same step.

writing

: Enable syntax checking of contracts but otherwise produce code exactly as if contracts where not present. The program behaves exactly as it did before contracts. This step enables initial coding and review.

logging

: Enable the evaluation of a contracts. When a contract is broken, execute a handler that logs the violation and then returns. Non-contract code remains exactly as if contracts where not present. The program behaves exactly as it did before contracts, except for the logging stream. This step enables identifying portions of the program that are out of contract and need fixing. John Lakos suggested this step.

terminating

: Enable the evaluation of a contracts. When a contract is broken, execute a handler that terminates the program, e.g. with `quick_exit`. Non-contract code remains exactly as if contracts where not present. The program behaves exactly as it did before contracts, except for termination. This step enables prevention of bad data being written to databases and the like.

exploiting

: Use unevaluated contracts as additional information in the optimization process. This step enables faster execution of known-good programs.

In order for adoption to be reasonable, **compilers must have options that control the extent to which they interpret and exploit contracts**. In particular, compilers need to refrain from exploiting contracts for optimization unless explicitly told to do

so.

Testing Contracts

How do we know contracts are properly specified? The first step is a thorough code review. The next step is testing. That is, we purposefully write code that violates the intended contract to see if the coded contract detects the violation. Again, this idea comes from John Lakos.

How do you test a behavior that leads to standard-undefined behavior?

- Rely on compiler-defined behavior. As with Adopting Contracts, we rely on a compiler that evaluates contracts and calls a handler, but otherwise does not alter the code generated.
- Rely on the nature of contract tests being shallow. This leads to two possible approaches.
    - Install a handler that does a long jump back to the test harness. Because the function under test is immediately beneath the harness, there is little opportunity for lost memory allocations between setting the long jump buffer and the jump itself.
    - Install a handler that throws an exception that caught in the harness. Again, there is little opportunity for losing memory allocations. However, the test must be compiled with an option that handles noexcept functions in the same way it would a throwing function.

Whatever approach is taken, **the test engineer needs a cooperative compiler**.

Managing Handlers

In the above discussion, we have described several different kinds of handlers. The question is when and how are these installed. Gabriel dos Reis shared Microsoft's view that handlers needed to be defined at compile time in order to prevent security problems with dynamically installed handlers. Note that **compile-time handlers makes the effect of a broken contract an attribute of the translation unit**.

Static handlers do not preclude dynamic handlers, as a static handler could call through a function pointer to a dynamically selected handler. However, in order for programmers setting the dynamic handler to reach all translation units that accept a dynamic handler, **the dynamic handler must be standard**.

Given several possible kinds of handlers, should there be a default? If so, what should it be? In the absence of any knowledge of the program development environment, the default should be to terminate the program quickly but safely. The mechanism for that termination is `quick_exit`.

Implementing Contracts

The two main issues in implementing contracts are effective dynamic evaluation and effective static evaluation.

Dynamic Evaluation

As discussed earlier, the primary implementation method is to evaluate any preconditions in the caller. However, that approach has two problems.

- For indirect calls, the caller does not know the callee and hence does not know the preconditions and hence cannot evaluate the preconditions. (We could potentially make conditions part of the type system, but doing so would be a substantial breaking change.)
- The evaluation of preconditions at call sites may substantially increase the size of the code due to code duplication.

Both problems are solved by providing multiple wrapper functions corresponding to the level of condition evaluation.

- evaluates the 'default' level of preconditions
- evaluates all preconditions
- evaluates critical preconditions and those not more complex than the function
- evaluates critical preconditions and those with constant complexity
- evaluates only critical preconditions
- evaluates no preconditions

The first wrapper would constitute the address of the function, and would be used for all indirect calls. The last wrapper would be used when the compiler chooses to evaluate the preconditions at the call site or when conditions are completely turned off at the call site. The other wrappers could be used with different compiler options.

It is important to note that **these wrappers become part of the C++ Application Binary Interface (ABI)**. The C++ committee and the compiler vendors need to have confidence that the contract management descriptions are useful and stable.

Note that this discussion omits the distinction between conditions and expectations. Since evaluating the latter essentially require maintaining execution statistics, it is likely that such evaluation will be conducted in conjunction with execution profiles, and hence should not affect the interfaces above.

Static Evaluation

Compilers routinely evaluate expressions with constant arguments. That technology applies readily to conditions.

Some compilers can use some expressions known to be true to inform the interpretation of subsequent expressions. However, we cannot say that such an ability is widely deployed or robust. However, any contract feature must enable the gradual evolution of compiler ability. So, it should neither require nor exclude expression propogation.

References

[GCC]
A GNU Manual, 6.58 Other Built-in Functions Provided by GCC, http://gcc.gnu.org/onlinedocs/gcc/Other-Builtins.html

[IEEE]
IEEE Standard for Floating-Point Arithmetic, IEEE Std 754-2008, http://ieeexplore.ieee.org/xpl/mostRecentIssue.jsp?punumber=4610933

Bibliography

Within this bibliography, entries marked with an asterisk indicate papers with a substantial survey component.

Ottosen
* Proposal to add Design by Contract to C++ N1613

Ottosen, Crowl, Widman
Proposal to add Contract Programming to C++ N1669 N1773 N1866 N1962
Contract Programming for C++0x N1800

Meredith
Addressing Exception Specifications for Next Generation of C++ N1825

Crowl, Ottosen
Synergies between Contract Programming, Concepts and Static Assertions N1867

Lakos, Zakharov, Beels, Myers
Centralized Defensive-Programming Support for Narrow Contracts N3604 N3753 N3877 N3963 N3997 N4075
Language Support for Runtime Contract Validation N4135
Language Support for Contract Validation N4253
Language Support for Contract Assertions N4378
FAQ about Contract Assertions N4379

Garcia
* Exploring the design space of contract specifications in C++ N4110

Krauss
Operator `assert` N4154

Krzemien'ski
* Value constraints N4160

Meredith
Library Preconditions are a Language Feature N4248

Garcia
C++ language support for contract programming N4293

Dos Reis, Lahiri, Logozzo, Ball, Parsons
Contracts for C++: What Are the Choices? N4319

Dos Reis, Garcia, Logozzo, Fähndrich, Lahiri
Simple Contracts for C++ N4415

Finkel
Generalized Dynamic Assumptions N4425

Brown
Proposing Contract Attributes N4435